# Edge cases (proving gather actually waits) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.


- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing


## Hint 1 — The idea, and the exact pieces

The exercise is really about one behavior of `asyncio.gather`: it runs its arguments concurrently, but it does not *return* until all of them have completed — its overall wall-clock time is bounded by whichever task takes longest, not whichever finishes first.

Both `fast_task` and `slow_task` use `await asyncio.sleep(...)` to simulate work, and both print a timestamp the moment they finish, before returning their value. The `main` coroutine records a start time, calls `await asyncio.gather(fast_task(), slow_task())`, then records and prints the time immediately after `gather` returns.

The exact pieces:

- `async def fast_task() -> str:` — `await asyncio.sleep(0.1)`, then print a timestamp, then return.
- `async def slow_task() -> str:` — same shape, `await asyncio.sleep(5)`.
- `time.perf_counter()` — printed at each finish point, and once right after `gather` returns.
- `await asyncio.gather(fast_task(), slow_task())` — the `await` here is what makes `main` pause until `gather` itself is done, which is exactly the 5 seconds set by `slow_task`, not the 0.1 seconds set by `fast_task`.

Think about what this means once "gather waits for the slowest" isn't 2 tasks but a group of 5 or 10 independent calls, and one of them is unusually slow — a flaky network path, an overloaded endpoint, whatever. Every fast task in that group finished its work long ago and is just sitting there, done, while the whole group's result stays locked behind the one slow straggler. The group's cost isn't its average task — it's its *worst* task, every single time.

The real design question isn't just "does gather wait for the slowest" (yes, always) — it's "should a task genuinely be allowed to take an unbounded amount of time inside a group I'm waiting on, or should something cap how long any one member of the group can hold up the rest?"

The extra piece that answers that question:

- **A per-task timeout, applied before the task ever reaches `gather`:** wrap each individual coroutine in `asyncio.wait_for(coro, timeout=...)` *before* passing it to `gather` — not around the whole `gather` call. That way, one genuinely hung task times out and fails on its own, instead of holding the entire group hostage indefinitely ( `asyncio.gather(..., return_exceptions=True)` from the `gather_speed` exercise is what stops that one timeout from losing the other results too).

## Hint 2 — The plan, and almost the whole thing

```
import asyncio
import time

define:
    async def fast_task() -> str:
        await asyncio.sleep(0.1)
        print(f"fast_task finished at {time.perf_counter():.2f}")
        return "fast"

define:
    async def slow_task() -> str:
        await asyncio.sleep(5)
        print(f"slow_task finished at {time.perf_counter():.2f}")
        return "slow"

define:
    async def main() -> None:
        start = time.perf_counter()
        print(f"starting at {start:.2f}")
        results = await asyncio.gather(fast_task(), slow_task())
        print(f"gather returned at {time.perf_counter():.2f}")
        print(results)

if __name__ == "__main__":
    asyncio.run(main())
```

Fill in the missing pieces yourself, then compare against the Solution.

Once that's working, add a task that's unbounded on purpose, and cap it individually:

```
make a third task, truly_slow_task, that sleeps for 60 seconds
(standing in for "might never come back")

wrap it individually, before gather ever sees it:
    bounded_slow = asyncio.wait_for(truly_slow_task(), timeout=2)

run gather(fast_task(), bounded_slow, return_exceptions=True):
    fast_task succeeds
    bounded_slow's slot holds a TimeoutError, not a hang
    -> the whole group finishes in ~2 seconds, not 60
```

```
# gather_practice.py — Edge cases section
async def truly_slow_task() -> str:
    await asyncio.sleep(60)
    return "finally done"

async def main_bounded() -> None:
    bounded = asyncio.wait_for(truly_slow_task(), timeout=2)
    results = await asyncio.gather(
        fast_task(), bounded, return_exceptions=True,
    )
    for result in results:
        if isinstance(result, Exception):
            print("a task timed out or failed:", result)
        else:
            print("a task succeeded:", result)
```

Time the whole `main_bounded()` call yourself to confirm it finishes in about 2 seconds, not 60, then compare against the Solution.

Full solution: Show me the solution